04 MAR 2026

# AADK TASK 3

Theme + Animation Integration: Building a Polished UI Module

MindMatrix

## 1. Module Definition

**Selected Module:**
Gallery of Task Cards with Expand/Collapse and Favourite.

**Description**

This module displays a list of task cards in my ZenTasks app. Each card can expand to show more details and can be marked as favourite with animated feedback.
Visually, this module represents the core interaction of the app. Functionally, it allows users to quickly scan tasks, interact smoothly, and get clear feedback through animation and theme consistency.

## 2. Data Model & State Variables

**Data Model**

```
data class TaskItem(
    val id: Int,
    val title: String,
    val description: String,
    val dueDate: String,
    val isFavourite: Boolean = false
)
```

**State Variables**

| State Variable | Purpose |
|---|---|
| isExpanded | Controls whether a card is expanded |
| isFavourite | Controls favourite icon animation and color |
| currentThemeMode | Light / Dark / Dynamic |
| animationProgress | Used for scale/alpha animation |

**Example:**

```
var isExpanded by remember { mutableStateOf(false) }
var isFavourite by remember { mutableStateOf(false) }
```

**Theme Token Mapping**

| UI State | Theme Token Used |
|---|---|
| Normal card background | colorScheme.surface |
| Expanded card | colorScheme.surfaceVariant |
| Favourite icon active | colorScheme.secondary |
| Favourite icon inactive | colorScheme.onSurfaceVariant |
| Title text | typography.headlineMedium |
| Description text | typography.bodyMedium |
| Card shape | shapes.medium |

## 3. UI Composition Plan

**Layout Structure**

```
ZenTasksTheme {
    Surface(
        color = MaterialTheme.colorScheme.background
    ) {
        LazyColumn {
            items(taskList) { task ->
                TaskCard(task)
            }
        }
    }
}
```

### TaskCard Composable Plan

```
Card(
    shape = MaterialTheme.shapes.medium,
    colors = CardDefaults.cardColors(
        containerColor = if (isExpanded)
            MaterialTheme.colorScheme.surfaceVariant
        else
            MaterialTheme.colorScheme.surface
    ),
    modifier = Modifier
        .fillMaxWidth()
        .animateContentSize()
)
```

## Animation Integration

### Expand/Collapse

- Trigger: Tap on card
- Animation: animateContentSize()
- Effect: Smooth height transition

### Favourite Icon Animation

Trigger: Tap heart icon
Animation:

- animateColorAsState() for color
- animateFloatAsState() for scale

### Example logic:

```
val iconColor by animateColorAsState(
    targetValue = if (isFavourite)
        MaterialTheme.colorScheme.secondary
    else
        MaterialTheme.colorScheme.onSurfaceVariant
)
val scale by animateFloatAsState(
    targetValue = if (isFavourite) 1.2f else 1f
)
```

### Animated Description Visibility

```
AnimatedVisibility(visible = isExpanded) {
    Text(
        text = task.description,
        style = MaterialTheme.typography.bodyMedium
    )
}
```

### Animation Triggers Summary

| Action | State Change | Result |
|---|---|---|
| Tap Card | isExpanded = !isExpanded | Card expands/collapses |
| Tap Favourite | isFavourite = !isFavourite | Icon scales + changes color |
| Switch Theme | currentThemeMode changes | Entire module re-colors smoothly |

## 4. Implementation Timeline & Dependencies

### Timeline Plan

Step 1
Set up theme tokens and ensure Light/Dark/Dynamic modes work.
Step 2
Build static UI layout with LazyColumn and TaskCard.
Step 3
Add animation triggers and integrate animate* APIs.
Step 4
Test performance, responsiveness, theme switching, and edge cases.

### Dependencies

| Library | Purpose |
|---|---|
| Coil | Load images (if task has image) |
| Material3 | Theming and components |
| Accompanist (optional) | Placeholder effects |

## 5. Quality, Accessibility & Performance Checklist

### Theme & Design Checklist

- No hardcoded colors
- All typography uses MaterialTheme.typography
- Shapes use MaterialTheme.shapes
- Proper spacing and padding

### Performance Checklist

- Efficient LazyColumn usage
- Avoid heavy animations in long lists
- Keep animation duration under 400ms
- Use stable keys in LazyColumn

### Accessibility Checklist

- Respect system reduce-motion settings
- Ensure text readable in dark mode
- Provide content descriptions for icons
- Maintain sufficient color contrast

**Edge Test Cases**

**Test Case 1**

Theme changes while card expansion animation is in progress.
Expected: UI should smoothly adapt colors without crashing or resetting state.

**Test Case 2**

User rapidly toggles favourite on a slow device.
Expected: Animation should remain smooth and not stack or freeze.

## 6. Reflection

The biggest challenge was balancing theme consistency with animation smoothness. I had to carefully ensure that animated values still used theme tokens instead of hardcoded values.
Combining theme and animation makes the UI feel alive and branded at the same time. Theme provides consistency, while animation provides interaction feedback.
However, developers should avoid adding animation everywhere. Too much motion can reduce clarity and impact performance. Animation should support functionality, not distract from it.